claude-windows / 96963dab-d38f-488c-af44-530ddbeefed1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\subagents\workflows\wf_dd531439-5e0\agent-adf0f1576d9058d21.jsonl`
- SHA-256: `fdca6780bc9761e193edf45403c0ddd341c99cb4ea3457cdf90690fd7b7428a7`
- Source modified: `2026-07-08T06:38:16+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_dd531439-5e0`
- session_id: `96963dab-d38f-488c-af44-530ddbeefed1`

Transcript

1. user (2026-07-08T06:36:06.556Z)

You are reviewing an UNCOMMITTED diff in an isolated git worktree at C:/Users/avidu/AppData/Local/Temp/claude/wt524-969 (branch feat/524-l4-primary, based on origin/master).
To see the change: cd to the worktree and run `git --no-pager diff` (working tree vs HEAD) and `git --no-pager diff --stat`; read the new files docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md and tests/test_video_id_trust.py directly. Read surrounding code as needed ? do NOT limit yourself to diff hunks.

CONTEXT ? GitHub issue #524 (epic): collapse the pipeline onto the L4 worker; shrink control-plane to a thin router. This PR delivers (1) a design doc resolving the compute-topology question (recommend co-located scale-to-zero Cloud Run Jobs + bucket upload; REJECT upload-onto-L4 and persistent-GPU with a cost model), and (2) "win #2": the control-plane drain trusts the SERVER-DERIVED submit-time md5 (jobs.video_id ? submit_time_video_id, #484 ? GCS metadata md5 converted to hex by backend/storage/gcs.py::_md5_hex) instead of re-hashing the fetched clip, gated by deployment.trust_submit_video_id (default True) and guarded by orchestration._looks_like_video_id (strict 32-char lowercase hex). Win #1 (validation-on-L4) is deliberately NOT in this PR ? it is delivered by the separate open branch feat/521-compile-on-l4 (phase_placement.validation); duplicating it is forbidden.

Report ONLY concrete, actionable findings with file:line. No praise, no restating the design. For each: what breaks, the exact failing input/state, and the fix. If a dimension is clean, say so explicitly.
A prior reviewer raised this finding on the SECURITY / TRUST BOUNDARY dimension:
  SEVERITY: nit
  FILE: docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md:41
  SUMMARY: The L6 'byte-identical video_id' claim silently omits its precondition (the bucket object is immutable in-flight), which is the load-bearing assumption for the whole win #2 trust argument.
  CLAIMED FAILURE: Line 41 states win #2 'produces a byte-identical video_id (the GCS md5Hash over the same bytes = compute_video_id)'. This equality only holds while the gs:// object is not overwritten between the submit-time stat and the drain fetch. The doc presents it as unconditional, so a future reader wiring a presigned direct-to-bucket upload (contemplated in the hybrid-C discussion) could reasonably assume win #2 stays safe when it would no longer be.
  PROPOSED FIX: Add one clause to L6 / the §Boundaries notes: the byte-identity holds because the input object is immutable in-flight in the current topology (server-mediated uploads, no client bucket-write creds, unique uuid upload paths); a future client-writable/presigned upload path MUST add generation-pinning (or a fetch-side md5 recheck) before enabling trust_submit_video_id, since the drain no longer re-hashes.

Adversarially VERIFY it against the real code/doc in C:/Users/avidu/AppData/Local/Temp/claude/wt524-969. Default to REFUTED unless you can construct the concrete failing input/state and confirm it in the actual code. A finding is CONFIRMED only if it is a real defect a maintainer must fix (not a style preference, not already-handled, not a

misread). State the verdict and the evidence (file:line you checked).

2. assistant (2026-07-08T06:36:10.089Z)

I'll verify this finding against the actual doc and code.

3. user (2026-07-08T06:36:19.135Z)

```
warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md', LF will
be replaced by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_video_id_trust.py', LF will be replaced by CRLF the
next time Git touches it
 backend/api/job_worker.py                          |   5 +
 backend/config/models.py                           |  10 +
 backend/orchestration.py                           |  51 ++++-
 .../L4_PRIMARY_TOPOLOGY.md                         | 225 ++++++++++++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/README.md           |   2 +
 tests/test_config_deployment.py                    |   2 +
 tests/test_video_id_trust.py                       | 173 +++++++++++++++
 7 files changed, 466 insertions(+), 2 deletions(-)
```

4. user (2026-07-08T06:36:20.061Z)

```
1       # L4-primary pipeline topology ? collapse the pipeline onto the worker, shrink the
control-plane to a thin router
2
3       > **Status:** `DRAFT` ? **owner-review** (recommends *against* the epic's proposed
direction #1; see §0/§8) · **Owner:** Avi · **Created:** 2026-07-08 · **Last updated:**
2026-07-08
4       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5       > **Issue:** [#524](https://github.com/Khelsutra/badminton-highlight-indexer/issues/524)
(epic) · **Builds on:** [#521](https://github.com/Khelsutra/badminton-highlight-
indexer/issues/521) (`phase_placement`) · **Motivated by:**
[#520](https://github.com/Khelsutra/badminton-highlight-indexer/issues/520) (the tracing that
surfaced the data-movement finding)
6       > **Relation to existing docs:** extends
[COMPUTE_DECOUPLED_SERVING/README.md](./README.md) (ADR-014) and [`PHASE_PLACEMENT.md`
(#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-
on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md); it does **not** supersede them ? it
answers the *topology* question they left open and sequences the migration on top of
`phase_placement`.
7       > **Honesty note:** each claim is tagged `[verified]` (checked against code/runlog),
`[researched]` (needs sign-off / external pricing I could not fetch live), or `[design]`
(proposed). Not legal/financial advice.
8
9       ---
10
11      ## 0. TL;DR
12
13      The 2026-07-08 live CUJ moved one 1.83 GiB clip **3×** (upload?bucket, bucket?control-
plane to validate, bucket?worker to detect) and pinned the CPU-only 2-vCPU control-plane for
**~16 min** of heavy work (validation 183 s + stitch ~12m55s). The epic proposes fixing this by
**uploading directly onto the L4** and async-copying to the bucket. **This design recommends
*against* that direction** and quantifies why: a same-region bucket?compute transfer inside GCP
is **LAN-fast and not billed as internet egress** `[researched]`, so the round-trip that upload-
onto-L4 saves is worth **~seconds and ~$0**, while keeping an L4 up to *receive* a **minutes-
long** upload burns **~175 s of idle GPU per run (~$0.08?0.16)** `[design]` and *opens* a
durability exposure window instead of closing one.
14
15      The cheaper, more durable, already-ratified path is the opposite: **keep the upload
streaming to the cheap bucket** (the #480 design ? the durable copy exists *before* the L4 ever
starts), and **collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** ?
which is exactly what **#521's `phase_placement` already wires as a config flip**. The residual
```

"~30 s pre-validation gap" is then killed not by moving the *upload* but by the control-plane **never downloading the file at all** (it delegates validation to the L4 per #521, and trusts the already-known submit-time md5 per this epic's win #2 ? no re-hash). Net data movement drops from 3× to the theoretical floor: **1 client upload + 1 LAN-fast intra-region pull**, with **zero idle-GPU cost** and **zero exposure window**.

Actionable now (this PR): the design doc + **win #2** (drop the redundant control-plane re-hash, config-guarded, reversible). **Win #1** (validation-on-L4) is **delivered by #521** ? this doc does not duplicate its knob. A persistent GPU (GCE VM / Cloud Run Service-with-GPU) is **rejected at current volume** on a quantified cost basis and left as an owner-gated future ADR.

1. Problem & goal

Why now. Tracing the live CUJ (`runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9 `[verified]`) produced two findings:

1. **Data is moved 3× per run** `[verified]`: (a) upload streams straight to `gs://?/uploads/?` (never touches control-plane disk ? the #480/#471 OOM fix, `backend/api/uploads.py`), computing the md5 incrementally as it streams; (b) the control-plane re-downloads the whole clip to scratch to validate ? `_execute_processing` ? `storage.acquire_local_input(gs://?)` at `backend/orchestration.py:748`, then a **redundant re-hash** `compute_video_id` at `:751`; (c) the worker downloads it a third time to detect (`[worker] pulled ?`). The unlogged **~30 s pre-validation gap** is finding (b).
2. **The control-plane is the scaling wall** `[verified]`: validation (183 s) **and** stitch (~12m55s, ~67 s/clip 4K re-encode, no NVENC) both run in-process on the 2-vCPU control-plane, serialized by `_LOCAL_COMPUTE_LANE` ? ~16 min of heavy CPU per run ? the API saturates past ~5 concurrent users.

The concrete outcome we want. The L4 worker becomes the primary compute for the whole pipeline; the control-plane shrinks to a **thin router** (auth · dispatch · DB · SPA + job status · signed-URL download) that does **no heavy decode/encode** and, ideally, **never pulls the full file**. Every move must be **reversible by config** with a **bucket-mediated fallback**, and land as small PRs on top of `origin/master`.

The hard question this doc must resolve (compute topology). "Upload directly to the L4" needs an **inbound endpoint on the GPU box**, but the current L4 is a Cloud Run **Job** (batch, no inbound HTTP ? `backend/compute/cloud_run.py`, dispatched per-request, `[verified]`). So the epic forces a choice between **A. Cloud Run Service-with-GPU**, **B. GCE L4 VM**, and **C. a hybrid** presign/redirect. §4 answers it with numbers.

Read to get current: [`PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md) ? the `phase_placement` knob this sequences on, [README.md](./README.md) §2 decisions **D4/D7/D16** (the ratified constraints that bound the options), and the runlog §9 (the evidence).

2. Decisions locked

#	Decision	Source / date	Implication
L1	**Keep upload ? bucket** (streaming, #480). The durable copy exists **before** any compute runs; the bucket *is* the "reliable co-located disk" the epic wants ? no VM needed for durability.	this doc `[design]` + #480 `[verified]`	No exposure window; upload cost stays on the cheap I/O path.
L2	**Collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** via `phase_placement` (#521), not onto a persistent GPU. Honors **D7** (scale-to-zero) + **D16(a)** (Jobs, not Services).	#521 `[verified]` + D7/D16	The "L4-primary" vision is realized by **config flips**, not a rearchitecture.
L3	**Kill the ~30 s gap by *not downloading to the control-plane*, not by uploading onto the L4.** Compose #521's validation-delegation (CP skips the validator suite) with win #2 (CP trusts the submit-time md5 ? no re-hash).	this doc `[design]`	The CP stops pulling the full file when validation is delegated; movement ? 3×?2×.
L4	**Reject persistent-GPU (GCE VM / Service-with-GPU) at current volume.** Breakeven vs scale-to-zero is a **~51 % on-demand / ~31 % spot duty cycle** (? 2,600?4,300

runs/mo sustained) `[design]`; we are at pilot volume. Revisit only behind a **new ADR** at sustained load. | §4.2 cost model `[design]` | Preserves per-run billing (D7/D8); avoids ~$382?637/mo idle. |
40 | L5 | **Reject "upload onto the L4."** It saves a ~free, seconds-fast intra-region LAN pull at the cost of ~175 s idle GPU/run **and** opens a durability window. | §4.3 `[design]` | The epic's proposed direction #1 is not adopted; owner sign-off requested (§8). |
41 | L6 | **Win #2 (drop the re-hash) is config-guarded + reversible** (`deployment.trust_submit_video_id`, default on) and produces a **byte-identical `video_id`** (the GCS `md5Hash` over the same bytes = `compute_video_id`). | win #2 `[verified]` | Zero-regression; one-flag rollback. |
42
43   *(IDs are `L#` to avoid colliding with README.md's `D#`. These refine ? never override ? D4/D7/D16.)*
44
45   ## 3. Foundation ? the ratified constraints this must live inside
46
47   The topology is **not a blank slate** ? ADR-014 (README.md §2) already ratified the serving substrate. The options in §4 are scored **against** these, and any option that breaks one needs a new ADR, not just this doc:
48
49 | Ratified decision | What it pins | Consequence for #524 |
50 |---|---|---|
51 | **D4** | In cloud-serving, **stitch runs on Cloud Run** (co-located, metered). | The live system's CP-stitch is a *deviation* from D4; #521 *realizes* D4. #524 is mostly **completing an already-ratified decision**, not inventing one. |
52 | **D7** | **Scale-to-zero, no warm pool** (a ~$500/mo idle GPU "would break per-video billing"). | Any persistent-GPU option (B, Service-with-min-instances) **contradicts D7** ? new-ADR territory (§4.2). |
53 | **D16(a)** | **Cloud Run Jobs** (not Services-with-GPU), **bucket-poll** for completion. | Option A (Service-with-GPU inbound upload) **contradicts D16(a)**. The Job model is load-bearing (reuses `JobWaiting`/`claim_due_job`). |
54 | **D16(b)** | Stitch encode = libx264 for parity; **NVENC deferred behind a knob**. | #521 lands exactly that knob (`cloud_stitch_encoder`), using this CUJ as the cost data D16(b) said to revisit on. |
55 | **D16(c)** | Input cap hard-reject `< 2 GB`. | The 1.83 GiB CUJ clip is *at* the cap; the topology must stay sane at 2 GB (upload minutes, pull seconds). |
56
57   **Key reframing this foundation gives us:** the owner's "reliable persistent disk" requirement is *already met* by the **regional GCS bucket** ? a durable, co-located, cheap "disk" the upload already writes to directly. The persistent-VM instinct solves a problem (durable local disk) that the bucket-first design doesn't have.
58
59   ## 4. Design / architecture ? resolving the topology question
60
61   ### 4.1 The measured pipeline (evidence base) `[verified]`
62
63   From runlog §9 (rev `rally-control-plane-00019`, "First CUJ.MP4", 1.83 GiB / 174.17 s / 4K-class):
64
65 | Phase | Wall | Machine | GPU needed? | Notes |
66 |---|---|---|---|---|
67 | Upload (client ? bucket) | **~2m55s** | client uplink | no | 1.83 GiB @ ~84 Mbps ? **owner bandwidth**, not infra |
68 | Accept ? validate start | ~30 s | control-plane | no | **the gap** = CP re-download to scratch (`acquire_local_input` + re-hash) |
69 | Validation (#512 one-pass) | **183 s** | control-plane 2 vCPU | no (CPU-bound) | scene_cut/blur/relevance shared decode |
70 | Dispatch ? worker ready | ~55 s | Cloud Run Job | ? | image pull + GCSFuse + torch/cuda init (cold start) |
71 | Detection (WASB) | **161.6 s** | L4 (cuda) | **yes** | the only genuinely GPU-bound phase |
72 | Ingest + finalize | ~4 s | control-plane | no | intervals ? DB |
73 | Compile / stitch | **~12m55s** | control-plane 2 vCPU | no (NVENC would need GPU) | 11× 4K re-encode+watermark |
74

Two facts that drive the whole design:
- **Only *detection* is GPU-bound.** Validation is CPU/decode-bound; stitch is encode-bound (NVENC *wants* a GPU but libx264 doesn't *need* one). So "collapse onto the L4" is about **co-locating CPU work next to the GPU on a box that's already up and already holding the bytes**, not about the GPU per se.
- **Upload dwarfs compute** (175 s upload vs 161.6 s GPU-detect). Anything that keeps a GPU allocated *during upload* roughly **doubles** the GPU-allocated time for zero compute.

4.2 Cost model (asia-southeast1, USD, Cloud Billing Catalog API, 2026-07-08) `[verified]` rates / `[design]` per-run

Rates pulled live from the Catalog API (`cloudbilling.googleapis.com`, project `gen-lang-client-0306026826`):

Resource	Rate	Source
Cloud Run **L4 GPU** (no zonal redundancy)	**$0.00022404 / s** = $0.806/hr	Catalog `[verified]`
Cloud Run vCPU (instance/jobs)	$0.0000216 / s	Catalog `[verified]`
Cloud Run memory	$0.0000024 / GiB·s	Catalog `[verified]`
? **L4 Job @ 8 vCPU / 32 GiB (running)**	**? $1.705 / hr** ($0.000474/s)	derived `[design]`
GCE **g2-standard-4** (4 vCPU/16 GiB + L4), on-demand	**$0.872 / hr** = **~$637 / mo**	Catalog `[verified]`
GCE g2-standard-4, **spot/preemptible**	$0.523 / hr = ~$382 / mo	Catalog `[verified]`
Cloud Run **GPU Service, warm** (min-instances=1, 4 vCPU/16 GiB)	? $1.256 / hr = **~$917 / mo** idle	derived `[design]`
Balanced PD	$0.11 / GiB·mo	Catalog `[verified]`

Per-run GPU-allocated cost (the term that varies by topology):

Topology	GPU-allocated time / run	GPU cost / run	? vs today	Concurrency wall?
Today (detect-only on L4; validate+stitch on CP)	~222 s (cold+detect+io)	**~$0.105**	?	**Yes** ? ~16 min CP CPU/run
#521 + win #2 (validate+detect+stitch on L4; upload?bucket)	~312 s (validate 60 + detect 162 + stitch 60 + io 30)	**~$0.148**	**+$0.04**	**No** ? CP is thin
Upload-onto-L4 (GPU up during upload too)	~487 s (+175 s idle upload)	**~$0.231**	**+$0.13**	No, but idle-GPU tax
Persistent GCE VM (on-demand)	billed 24/7 regardless	**$637/mo flat**	breaks per-run billing	No

Breakeven (persistent vs scale-to-zero): a persistent on-demand g2 ($0.872/hr) beats the scale-to-zero Job ($1.705/hr *while running*) only at a **duty cycle ? $0.872/$1.705 ? 51 %** (spot: ? 31 %). At ~312 s/run that is **~2,600?4,300 runs/month sustained, 24/7** `[design]`. Pilot volume is *handfuls/day*. **Scale-to-zero wins by ~2 orders of magnitude** and preserves per-run billing (D7/D8). A persistent GPU is also **idle >95 % of the time** at our volume ? the ~$382?637/mo would be almost pure waste.

4.3 The three topology options, scored

Option A ? Cloud Run Service-with-GPU hosting upload + pipeline.
- *Latency:* scale-to-zero ? same ~55 s cold start as the Job; **min-instances=1** removes cold start but pins a **~$917/mo warm idle GPU**.
- *Cost:* to accept the upload, the GPU Service instance must be **up for the whole 175 s upload** ? the idle-GPU tax of §4.2 on *every* request, plus the warm-idle bill if min-instances>0.
- *Ops/parity:* **contradicts D16(a)** (Jobs, not Services) and **D7** (scale-to-zero) ? abandons the `JobWaiting`/`claim_due_job`/bucket-poll machinery that's shipped and tested. A Service also needs request-timeout/concurrency tuning the batch Job sidesteps.
- **Verdict: reject.** Buys an inbound endpoint we don't need (the bucket is the inbound endpoint) at the cost of the idle-GPU tax + two ratified-decision reversals.

Option B ? GCE L4 VM (persistent, reliable PD).

- *Reliability:* real, attached, reliable PD ? the owner's stated appeal. But **the bucket already gives us durable co-located storage** (L1), so the PD solves a non-problem.
- *Cost:* **$382?637/mo flat**, >95 % idle at pilot volume; breaks per-run billing (D7/D8). Spot is cheaper but **preemptible** ? directly against the "reliable" motivation.
- *Ops:* we now **own a VM** ? patching, disk-fill, crash-recovery, autoscaling-group if we ever need >1. The Job model has none of this.
- **Verdict: reject at current volume; owner-gated future ADR** if sustained load crosses the §4.2 breakeven.

Option C ? Hybrid: control-plane presigns/redirects the upload to a just-spun-up L4, pipeline runs there, async copy to bucket.

- *The fatal timing problem:* compute needs the **whole** file, so the L4 cannot start useful work until the upload finishes. Spinning the L4 up *to receive* the upload means the GPU is **allocated and idle for the entire 175 s upload** ? the §4.2 idle tax, same as A. Upload and compute **cannot overlap**.
- *Durability:* introduces an **exposure window** (upload-complete ? async-mirror-confirmed) where the only copy is on L4 local disk; a preemption/crash in that window **loses the source** unless the client re-uploads (see §4.5).
- **Verdict: reject.** It pays the idle-GPU tax *and* takes on a durability window, to save a transfer that is ~free and seconds-fast.

Option A? (recommended) ? co-located Jobs, bucket-mediated upload (the L1?L3 design).

- Keep **upload ? bucket** (cheap, bounded-memory, durable-immediately). Keep the **scale-to-zero L4 Job**. Move **validate + stitch** onto the Job via `phase_placement` (#521). Make the CP **stop downloading** (delegate validation + trust the md5, win #2). The Job pulls the file **once** from the same-region bucket at LAN speed.
- *Data movement:* **1 client upload + 1 intra-region LAN pull** ? the floor. (Detection and validation/stitch all run in **one** Job execution, so it's one pull, not two.)
- *Cost:* +$0.04/run GPU vs today, **?~16 min CP CPU/run** (the actual scaling win). No idle tax, no warm bill, no VM.
- *Durability:* **no exposure window** ? the bucket copy exists before the Job starts.
- *Ops/parity:* **honors D4/D7/D16**; reuses the shipped dispatch/poll path; every phase is a **config flip with an in-process fallback** (#521's `_dispatch_*_remote` already logs-and-falls-back to in-process when the box isn't cloud-wired).

> **The one-line intuition:** the bucket *is* the co-located reliable disk; the GPU should attach only for the ~5 min of compute, pulling from it at LAN speed ? never sit idle for the ~3 min upload.

4.4 The minimal control-plane surface (the "thin router")

After A?, the control-plane does **only** I/O-light, decode-free work ? trivially horizontally scalable (the one caveat is SQLite; a Postgres move is out of scope, noted in §6):

Keep on the control-plane	Why it must stay	Code anchor `[verified]`
Cloudflare-Access edge auth (verify `Cf-Access` JWT)	D9 ? one identity system; direct-hit spoofing guard	`backend/api/auth.py`, gated at `/api/process`
Upload endpoint (streams parts ? bucket)	Bounded-memory, no decode (#480); the durable-copy producer	`backend/api/uploads.py`
Dispatch / routing (submit job, dispatch Cloud Run Job, bucket-poll)	Owns the async job lifecycle (`JobWaiting`/`claim_due_job`)	`backend/orchestration.py`, `backend/compute/cloud_run.py`
DB writes (jobs, videos, `play_segments`, validation verdict)	The control-plane owns the SQLite DB (worker uses a throwaway scratch DB)	`backend/infrastructure/database*.py`
SPA serving + job status (`/api/jobs/{id}` polling)	The UX surface	`backend/api/jobs.py`, `backend/ui/`
Signed-URL download (`/api/highlights` 307)	Reel is already in the bucket; CP just signs	`backend/main.py::get_highlights`
~~Validation~~	? **L4** via `phase_placement.validation` (#521)	delegated
~~Stitch/compile~~	? **L4** via `phase_placement.compile` (#521)	delegated
~~Full-file download + re-hash~~	? **eliminated** (delegate validation + trust

md5, win #2) | this PR |

    ### 4.5 The async copy-to-bucket durability model (spec ? needed *only* if a future upload-onto-L4 path is ever adopted)

    The recommended A? design **has no async-copy step** (the bucket copy is the *upload*, produced before compute). This section specs the model the epic asked for, so that if the owner overrides §8 and adopts upload-onto-L4, the durability semantics are pinned rather than improvised:

    - **When the mirror runs:** kick a background `storage.put(local ? gs://?/uploads/?)` **immediately on upload-complete** (not "during segmentation" ? starting it later only lengthens the exposure window; there's no compute reason to delay it). Run it concurrently with validation/detect.
    - **Failure/resume:** retry with capped exponential backoff; the mirror is **idempotent** (same object key = the md5). On worker preemption before the mirror confirms, the run is **lost** ? the job must fail with a *retryable* status so the client can re-submit (the source is still on the client). **Never** report success until the mirror is **confirmed** (object exists + size/md5 match).
    - **Exposure window:** upload-complete ? mirror-confirmed, the **only** durable copy is L4 local PD. Window length ? mirror wall (1.83 GiB intra-region ? seconds?tens of seconds `[design]`). A preemption in-window = re-upload. **This window is the entire reason A? is preferred** ? A?'s window is *zero*.
    - **Ack contract:** the client keeps the source until the API returns `mirror_confirmed:true`; only then may it release it. (A? needs no such contract ? bucket-first.)

    ## 5. Phased migration ? every step a config flip with a bucket-mediated fallback

    Built on #521's `resolve_phase_placement(cfg, phase)` so **placement is config, not code**. Each phase is independently reversible (`control_plane` ? `cloud_run_l4`) and falls back to in-process on a non-cloud box.

Phase	Change	Mechanism	Fallback	Status
P0	*Today* ? detect on L4; validate+stitch on CP	`compute_target=cloud_run`	in-process	shipped `[verified]`
P1	**Stitch ? L4**	`phase_placement.compile=cloud_run_l4` (+ `cloud_stitch_encoder`)	in-process stitch	**#521 (open)**
P2a	**Drop the CP re-hash** (trust submit-time md5)	`deployment.trust_submit_video_id` (default on)	re-hash on the drain	**this PR (win #2)**
P2b	**Validation ? L4** (CP stops running validators)	`phase_placement.validation=cloud_run_l4` (guards: requires cloud segment)	CP validates	**#521 (open)**
P2 net	**CP never downloads the file** ? the ~30 s gap dies; movement 3×?2×	P2a ? P2b together	either half alone degrades gracefully	needs #521 **+** win #2 merged
P3	Worker serializes its **validation verdict** back (per-validator results + #518 cache warm), so a worker-side reject renders the same SPA detail as a CP reject	new `report.json`/marker field + CP ingest	worker reject = rc?0 + message (today)	**follow-up** (see §6)
P4	*If and only if* sustained load crosses the §4.2 breakeven	persistent worker (VM/Service)	scale-to-zero Job	**owner-gated, new ADR**

    The "collapse onto the L4" the epic asks for = P1 + P2, both config flips on #521's knob + this PR's win #2. No rearchitecture, no new infra, fully reversible.

    ## 6. Honest limits ? what this does NOT do / does NOT prove

    - **A green suite does not prove the cloud path** ? the tests are fully-mocked/offline (a fake `CloudRunJobClient`). P2's "CP never downloads" and the idle-GPU/round-trip numbers are validated **in the cost model + code paths**, not on live infra. A live CUJ on rev ? `00020` (owner-side, D17 budget) is required to *confirm* the gap is gone and the movement is 2×.
    - **Network-egress "free intra-region" is `[researched]`, not live-verified** ? general web egress is blocked here, so same-region GCS?Cloud Run being un-billed-as-internet-egress rests on documented GCP behavior, not a fetched invoice. If it were *not* free, A? is *still*

cheapest (it moves the fewest bytes); the conclusion is robust to this uncertainty.
178   - **Win #2 changes nothing when no trusted id is present** ? local/appliance uploads and non-GCS direct-URI submits (`submit_time_video_id` ? `None`) still re-hash. The win applies to the GCS cloud-serving path (the CUJ case).
179   - **P3 (worker validation-verdict serialization) is a real gap #521 leaves open** `[verified]`: today a worker-side validation reject surfaces as `rc?0` + a message, but the per-validator `ValidationResult` list (`result['results']` the SPA renders) and the #518 `validation_cache` warm are **not** reconstructed on the CP. Until P3, moving validation to the L4 (P2b) trades richer verdict UX + cache-warming for the CPU offload. Called out so P2b isn't flipped on blind.
180   - **SQLite is the residual CP scaling limit** ? the thin router is stateless *except* the SQLite DB; true horizontal scale-out needs Postgres (out of scope; noted).
181   - **The #513 asymmetry reverses under P2b** `[verified]`: worker-side validators run at model-default thresholds unless the CP forwards `validation.*` as `--set`. #521's validation-delegation path must forward those (or re-inherit the divergence that caused the #513 incident). Flagged for the #521 review, not owned here.
182
183   ## 7. Deliverables & progress tracker ? source of truth
184
185   Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, branched from `origin/master`, no stacking.
186
187 | ID | Deliverable | Depends on | Gated? | Status | PR |
188 |----|-------------|-----------|--------|--------|----|
189 | P0 | **This design doc** (resolves topology A/B/C; recommends A?; specs durability + phased migration) | ? | owner-review | ? (this PR) | #524 |
190 | P1 | **Win #2** ? drop the CP re-hash; trust submit-time md5; `deployment.trust_submit_video_id` (default on), reversible | ? | No | ? (this PR) | #524 |
191 | P2 | **Win #1** ? validation-on-L4 (`phase_placement.validation`) | #521 | No | ? **delivered by #521** ? not duplicated here | #521 |
192 | P3 | **CP-download-avoidance measured live** ? confirm the ~30 s gap is gone + movement is 2× on rev ? `00020` | #521 + P1 merged | ? owner-side (live spend, D17) | ? | ? |
193 | P4 | **Worker validation-verdict serialization** (§6 gap) so P2b keeps rich SPA verdict + #518 cache | #521 | No | ? follow-up | ? |
194 | P5 | **Persistent-GPU re-evaluation** only if load crosses §4.2 breakeven | volume | ? new ADR | ? | ? |
195
196   ## 8. Open questions ? owner / external
197
198   **Blocking (owner sign-off):**
199   1. **Do you accept rejecting "upload directly onto the L4" (L5)?** The epic proposed it; this doc recommends against it with the §4.2/§4.3 idle-GPU-vs-free-LAN-pull numbers. A? delivers the same "collapse onto the L4" outcome, cheaper and more durable, via config. *If you still want upload-onto-L4* (e.g. for a future no-bucket edge deployment), §4.5 specs its durability model ? but it needs a Service-with-GPU or VM (a D16(a)/D7 reversal ? new ADR).
200   2. **Flip P2b (validation?L4) before or after P3 (verdict serialization)?** P2b alone offloads CPU but degrades the reject-UX + skips #518 cache-warm (§6). Recommend: land P3 first, then P2b.
201
202   **Nice-to-know:**
203   3. Is pilot volume expected to cross the §4.2 breakeven within the planning horizon? (Decides whether P5 is ever live.)
204
205   ## 9. Definition of done
206
207   - [x] Design doc resolves the topology question (A/B/C scored) with a clear recommendation (A?) + quantified cost/latency/reliability. `[this PR]`
208   - [x] Minimal control-plane surface defined (§4.4). `[this PR]`
209   - [x] Async copy-to-bucket durability model specified with failure/resume + exposure window (§4.5). `[this PR]`
210   - [x] Phased migration laid out on #521's `phase_placement`, each step a config flip with a bucket-mediated fallback (§5). `[this PR]`
211   - [x] Win #2 landed (config-guarded, reversible, tests green). `[this PR]`
212   - [ ] Win #1 confirmed delivered by #521 (not duplicated). `[cross-PR]`
213   - [ ] Live CUJ confirms the gap is gone + movement is 2× (P3, owner-side). `[gated]`

```
214     - [ ] Owner signs off on L5 (reject upload-onto-L4) ? flip Status ? IN PROGRESS/DONE.
215
216     ## 10. References
217
218     **Internal code anchors `[verified]`:** `backend/api/uploads.py` (streamed md5),
`backend/orchestration.py:220-239` (`submit_time_video_id`, #484) / `:741-763` (re-hash) /
`_maybe_dispatch_remote`, `backend/api/jobs.py:80-153` (submit-time cache already trusts the
md5), `backend/api/job_worker.py:122-193` (`_job_to_request`/`_run_claimed_job`),
`backend/compute/cloud_run.py` (dispatch + bucket-poll), `backend/config/placement.py` (#521
`resolve_phase_placement`), `backend/worker/__main__.py` (worker validation + `--skip-
validation`).
219     **Internal docs:** [`PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-
highlight-indexer/blob/feat/521-compile-
on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md), [README.md](./README.md) (ADR-014,
D4/D7/D16), `runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9
(evidence).
220     **External `[verified]` rates:** GCP Cloud Billing Catalog API
(`cloudbilling.googleapis.com`, asia-southeast1, USD, 2026-07-08). **`[researched]`:** same-
region GCS?Cloud Run egress treatment (not live-fetchable here).
221
222     ---
223
224     ### Changelog
225     - `2026-07-08` ? initial draft: topology A/B/C resolved (recommend A? = co-located Jobs
+ bucket upload; reject persistent-GPU + upload-onto-L4 with a cost model); durability + phased
migration on #521's `phase_placement`; win #2 landed alongside.
226
```

5. assistant (2026-07-08T06:36:25.977Z)

Let me examine the relevant code paths to verify the trust boundary claim.

6. user (2026-07-08T06:36:27.377Z)

```
200         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201             return True
202         has_job = bool(
203             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204         )
205         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206         return bool(has_job and out_root)
207
208
209     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
211         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
212         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
213         (queue/cold-start), so the very first tick already shows movement."""
214         minutes = int(elapsed_sec) // 60
215         if minutes < 1:
216             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
217         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
218
219
220     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
225         if getattr(input_ref, "backend", "") != "gcs":
```

```
            return None
        sc = _storage_settings(cfg)
        if not sc:
            return None
        try:
            st = storage.get_storage("gcs", config=sc).stat(input_ref)
            return getattr(st, "md5", None) or None
        except Exception:  # noqa: BLE001 ? best-effort identity, the drain still hashes
            logger.info(
                "[API] submit-time md5 unavailable for %s (metadata stat failed)",
                getattr(input_ref, "uri", input_ref),
                exc_info=True,
            )
            return None


    def _looks_like_video_id(value: Optional[str]) -> bool:
        """True iff ``value`` is a well-formed ``video_id`` ? the 32-char lowercase hex MD5
that
        ``compute_video_id`` produces (`backend/utils/hashing.py`). Guards the #524 win-#2
trust path:
        a trusted id is only reused in place of a re-hash when it has the EXACT shape the
drain would
        have computed, so a malformed / truncated / wrong-encoding id can never become the
DB key ?
        it degrades to a real hash instead. ``submit_time_video_id`` already normalizes the
gs://
        base64 md5 to this hex form (#484, `backend/storage/gcs.py::_md5_hex`)."""
        return (
            isinstance(value, str)
            and len(value) == 32
            and all(c in "0123456789abcdef" for c in value)
        )


    def _cached_process_payload(
        video_id: str, validation_results, segments, requested_compute
    ) -> Dict[str, Any]:
        """The /api/process result for a video whose detection already exists ? ONE shape
for
        both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
        return {
            "video_id": video_id,
            "status": "success",
            "message": (
                f"Reused {len(segments)} cached play segments. "
                "Pass force=true to reprocess this video."
            ),
            "results": validation_results or [],
            "play_segments": segments,
            "cached": True,
            "compute": {"path": "cached", "requested": requested_compute},
        }


    def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
        """Fetch + parse one small JSON object (cloud backends require a real dst ? the
        ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
        ref = storage.parse_uri(uri)
        with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
            local = storage.fetch(
                uri, os.path.join(td, ref.name or "report.json"), config=storage_config
            )
```

```
283             with open(local, encoding="utf-8") as f:
284                 loaded = json.load(f)
285         return loaded if isinstance(loaded, dict) else None
286
287
288     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
289     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
290     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
291     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
292     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
293     _REHYDRATE_LOCKS_GUARD = threading.Lock()
294
295
296     def _rehydrate_lock(video_id: str) -> threading.Lock:
297         with _REHYDRATE_LOCKS_GUARD:
298             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
299
300
301     def probe_process_cache(db, cfg, video_id: str) -> bool:
302         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
303
304         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
305         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
306         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
307         errors report False (? the normal dispatch path with its guards)."""
308         try:
309             existing = db.get_video(video_id)
310             if (
311                 existing
312                 and existing.get("status") == "processed"
313                 and db.query_play_segments(video_id, {})
314             ):
315                 return True
316             sc = _storage_settings(cfg)
317             if not sc or (sc.get("backend") or "local") == "local":
318                 return False
319             output_root = (sc.get("output_root") or "").rstrip("/")
```

7. user (2026-07-08T06:36:34.123Z)

```
diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
index a90b14e..f8da26b 100644
--- a/backend/api/job_worker.py
+++ b/backend/api/job_worker.py
@@ -173,6 +173,11 @@ def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
                 db,
                 req,
                 progress=lambda stage: db.set_job_progress(job_id, stage),
+                # #524 win #2: hand the drain the submit-time md5 already on the job row
+                # (``submit_time_video_id``, #484 ? server-derived, byte-identical to
+                # compute_video_id) so it can skip re-hashing the fetched clip. "" ? unknown
+                # (local upload / non-gcs URI) ? the drain hashes as before.
+                trusted_video_id=job.get("video_id") or None,
             )
         if result.get("video_id"):
             db.set_job_video_id(job_id, result["video_id"])
diff --git a/backend/config/models.py b/backend/config/models.py
index a7c997c..71e43b8 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
```

```
@@ -481,6 +481,16 @@ class DeploymentConfig(BaseModel):
     # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect + queue/cold-
start,
     # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid GPU run.
     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
+    # Trust the SUBMIT-TIME md5 instead of re-hashing the fetched file on the drain (#524 win
#2).
+    # The chunked upload already streams the whole-file md5 (``backend/api/uploads.py``), and
the
+    # submit hook records it on the job row (``submit_time_video_id``, one gs:// metadata stat
? #484);
+    # that value is BYTE-IDENTICAL to ``compute_video_id`` (same bytes, same MD5) and is
ALREADY
+    # trusted for the submit-time cache probe (``backend/api/jobs.py``). When True (default)
the drain
+    # reuses it and SKIPS re-hashing the multi-GB clip it just fetched to scratch; when the id
is absent
+    # (a local/appliance upload or a non-gcs direct-URI submit ? ``submit_time_video_id``
returns None)
+    # the drain still hashes, exactly as before. Set False to force a re-hash unconditionally
+    # (belt-and-suspenders rollback ? reverts to pre-#524 behaviour with no redeploy).
+    trust_submit_video_id: bool = True


 class BillingConfig(BaseModel):
diff --git a/backend/orchestration.py b/backend/orchestration.py
index 8b7851e..7769e4e 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -239,6 +239,20 @@ def submit_time_video_id(input_ref, cfg) -> Optional[str]:
         return None


+def _looks_like_video_id(value: Optional[str]) -> bool:
+    """True iff ``value`` is a well-formed ``video_id`` ? the 32-char lowercase hex MD5 that
+    ``compute_video_id`` produces (`backend/utils/hashing.py`). Guards the #524 win-#2 trust
path:
+    a trusted id is only reused in place of a re-hash when it has the EXACT shape the drain
would
+    have computed, so a malformed / truncated / wrong-encoding id can never become the DB key ?
+    it degrades to a real hash instead. ``submit_time_video_id`` already normalizes the gs://
+    base64 md5 to this hex form (#484, `backend/storage/gcs.py::_md5_hex`)."""
+    return (
+        isinstance(value, str)
+        and len(value) == 32
+        and all(c in "0123456789abcdef" for c in value)
+    )
+
+
 def _cached_process_payload(
     video_id: str, validation_results, segments, requested_compute
 ) -> Dict[str, Any]:
@@ -706,7 +720,7 @@ def _maybe_dispatch_remote(


 def _execute_processing(
-    config=None, db=None, req=None, progress=None
+    config=None, db=None, req=None, progress=None, trusted_video_id=None
 ) -> Dict[str, Any]:
     """The full hash ? validate ? segment ? report pipeline for one video.

@@ -716,6 +730,15 @@ def _execute_processing(
     test code that calls ``_execute_processing(req)`` still works.

     ``progress`` is an optional callable(stage: str) used to surface coarse job progress.
```

```
+
+    ``trusted_video_id`` is the SERVER-DERIVED submit-time md5 the drain already knows for this
+    job (``jobs.video_id`` ? ``submit_time_video_id``, #484) ? byte-identical to
+    ``compute_video_id`` and already trusted for the submit-time cache probe. When present (and
+    ``deployment.trust_submit_video_id`` is on), it is reused so the drain SKIPS re-hashing the
+    multi-GB clip it just fetched (#524 win #2); absent / malformed ? the drain hashes as
before.
+    It is a keyword-only, internal channel ? NOT a client-supplied request field (a caller's
+    ``/api/process`` body cannot inject it; the submit hook recomputes the authoritative id).
+
     Raises on unexpected internal errors ? the caller records those as a job failure
     """
     import backend.main as _main  # call-time seam: main's globals/model stay patchable
@@ -748,7 +771,31 @@ def _execute_processing(
         local_video = storage.acquire_local_input(
             req.video_path, getattr(config, "storage", None)
         )
-        video_id = _main.compute_video_id(local_video)
+        # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing the whole
+        # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
+        # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the submit-
time
+        # cache probe trusts, so reusing it here is consistent, not a new trust assumption.
Gated by
+        # deployment.trust_submit_video_id (default on) and only honoured when the id is well-
formed
+        # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The fetch
above
+        # is unchanged; only the redundant hash is skipped.
+        trust_submitted = bool(
+            getattr(
+                getattr(config, "deployment", None), "trust_submit_video_id", True
+            )
+        )
+        # STRICT: trust only an id in the EXACT form the drain would have produced (32-char
+        # lowercase hex, per compute_video_id / submit_time_video_id). No normalization ? a
+        # differently-cased or malformed id is a signal something is off, so re-hash rather
than
+        # silently coerce it into the DB key.
+        candidate = trusted_video_id or ""
+        if trust_submitted and _looks_like_video_id(candidate):
+            video_id = candidate
+            logger.info(
+                "[API] trusting submit-time video_id=%s ? skipped the drain re-hash (#524).",
+                video_id,
+            )
+        else:
+            video_id = _main.compute_video_id(local_video)
     except Exception:
         storage.cleanup_acquired_local_input(
             req.video_path, local_video, getattr(config, "storage", None)
```

8. assistant (2026-07-08T06:36:40.514Z)

Let me examine the upload path to verify the immutability-in-flight assumption.

9. user (2026-07-08T06:36:41.524Z)

```
1       """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3       Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4       proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2). Where they assemble depends
5       on the input backend (CONTROL_PLANE_DURABLE_REBUILD P2):
6
7       * ``storage.input_backend == "local"`` (appliance/default): parts append to a partial
```

```
        file under security.upload_dir, exactly as before.
    * a remote input backend with an ``input_root`` (the hosted alpha: gcs): parts STREAM
        through a ``storage.open_writer`` into ``<input_root>/uploads/<upload_id>/<name>``
        while an incremental MD5 accumulates ? the assembled file never touches local
        disk/tmpfs. The 4 GiB RAM-``/tmp`` control plane OOM'd on a 2.68 GB browser upload
        (#480/#471); after this, instance memory bounds a CHUNK, not the video. ``complete``
        returns the staged URI (inside the #465 input jail by construction ? it is under
        ``input_root``) plus the streamed md5 as ``video_id``.

    Upload state is in-process by design (this is the single-box alpha): a server restart
    aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
    arrives with PR-3 identity scoping.

    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
    reads the live value the server set at startup (and so tests that monkeypatch
    ``backend.main.global_config`` flow through unchanged).
    """

    import hashlib
    import os
    import threading
    import unicodedata
    import uuid
    from typing import Any, Dict, Optional

    from fastapi import APIRouter, HTTPException, Request
    from pydantic import BaseModel
    from starlette.concurrency import run_in_threadpool

    from backend import storage
    from backend.utils.app_home import (
        resolve_output_dir,  # P0a: app-home-aware path resolution
    )
    from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
    from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
    from backend.utils.paths import safe_output_filename

    router = APIRouter()

    _uploads: Dict[str, Dict[str, Any]] = {}
    _uploads_lock = threading.Lock()


    class UploadInitRequest(BaseModel):
        filename: str
        total_size_bytes: Optional[int] = None


    class UploadCompleteRequest(BaseModel):
        total_parts: int


    def _upload_cfg():
        # Resolved lazily (import inside the function) to read the LIVE startup config that
        # main() sets on the module global, and to avoid a circular import at module load
        # (main.py imports this router; this module must not import main at import time).
        import backend.main as _main

        sec = getattr(_main.global_config, "security", None)
        upload_dir = resolve_output_dir(
            getattr(sec, "upload_dir", None) or "output/uploads"
        )  # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
```

```
        exts = [
            e.lower().lstrip(".")
            for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
        ]
        return upload_dir, max_bytes, part_bytes, exts


    def upload_caps(cfg) -> Dict[str, int]:
        """The upload limits /api/capabilities advertises (#480 item 5, CP-rebuild P5a).

        ``max_upload_mb`` is what the SPA's pre-flight guard rejects oversize files against
?
        without it the guard is dead code and the user learns from a mid-upload 413.
        ``max_part_mb`` is informational: each /api/upload/init response carries the
        authoritative value the client actually chunks by."""
        sec = getattr(cfg, "security", None)
        return {
            "max_upload_mb": int(getattr(sec, "max_upload_mb", 4096) or 4096),
            "max_part_mb": int(getattr(sec, "max_part_mb", 95) or 95),
        }


    def _staging_target() -> "Optional[Dict[str, Any]]":
        """The remote staging root + storage settings, or None for the local-file flow.

        Remote staging is selected by the same input config /api/process resolves against
        (storage.input_backend + input_root), so a staged upload is submittable verbatim.
        """
        import backend.main as _main

        sc = getattr(_main.global_config, "storage", None)
        if sc is None:
            return None
        input_backend = getattr(sc, "input_backend", "local") or "local"
        input_root = (getattr(sc, "input_root", "") or "").rstrip("/")
        if input_backend == "local" or not input_root:
            return None
        cfg = sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
        return {"root": input_root, "config": cfg}


    def _abort_upload(upload_id: str) -> None:
        with _uploads_lock:
            st = _uploads.pop(upload_id, None)
        if not st:
            return
        writer = st.get("writer")
        if writer is not None:
            # A streaming writer has no cancel ? close() finalizes whatever was staged, then
            # the partial object is deleted. Both are best-effort: the upload is already
            # aborted from the client's point of view.
            try:
                writer.close()
            except Exception:  # noqa: BLE001 ? abort cleanup must never mask the abort
                pass
            try:
                ref = storage.parse_uri(st["stage_uri"])
                storage.get_storage(ref.backend, config=st.get("config")).delete(ref)
            except Exception:  # noqa: BLE001
                pass
            return
        try:
            os.remove(st["tmp_path"])
        except OSError:
            pass
```

```
137
138
139     def _normalize_upload_filename(filename: str) -> str:
140         """Normalize browser-provided file names before extension validation.
141
142         Some mobile/file-provider pickers can hand us visually normal names with
compatibility
143         characters (for example a fullwidth dot) or trailing whitespace/dots. Keep the
existing
144         traversal/null-byte checks in ``safe_output_filename`` as the authority after this
cleanup.
145         """
146         return unicodedata.normalize("NFKC", filename).strip().rstrip(". ")
147
148
149     @router.post("/api/upload/init")
150     def upload_init(req: UploadInitRequest):
151         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
152         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
153         try:
154             name = safe_output_filename(_normalize_upload_filename(req.filename))
155         except ValueError as e:
156             raise HTTPException(status_code=400, detail=str(e)) from e
157         ext = os.path.splitext(name)[1].lower().lstrip(".")
158         if ext not in exts:
159             raise HTTPException(
160                 status_code=400,
161                 detail=f"Extension '.{ext}' not allowed. Allowed: {sorted(exts)}",
162             )
163         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
164             raise HTTPException(
165                 status_code=413,
166                 detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
167             )
168         upload_id = uuid.uuid4().hex
169         staging = _staging_target()
170         if staging is not None:
171             # Remote staging: stream parts straight into the bucket under the input root ?
172             # never assembled on local disk/tmpfs (CONTROL_PLANE_DURABLE_REBUILD P2). The
173             # upload_id in the key keeps concurrent same-name uploads distinct, and the
free-
174             # space guard is skipped: no local bytes are consumed.
175             stage_uri = f"{staging['root']}/uploads/{upload_id}/{name}"
176             try:
177                 writer = storage.open_writer(stage_uri, config=staging["config"])
178             except NotImplementedError:
179                 staging = None  # backend can't stream (e.g. mounted-Drive) ? stage locally
180             else:
181                 with _uploads_lock:
182                     _uploads[upload_id] = {
183                         "filename": name,
184                         "writer": writer,
185                         "hasher": hashlib.md5(),
186                         "stage_uri": stage_uri,
187                         "config": staging["config"],
188                         "parts": 0,
189                         "bytes": 0,
190                     }
191                 return {
192                     "upload_id": upload_id,
193                     "max_part_mb": part_bytes // (1024 * 1024),
194                     "next_part": f"/api/upload/{upload_id}/part/0",
195                 }
196         os.makedirs(upload_dir, exist_ok=True)
197         # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
```

```
effort ? an
198         # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
(never blocks a
199         # legit upload); the per-part total cap in upload_part is the hard backstop either
way.
200         require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
201         tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
202         open(tmp_path, "wb").close()
203         with _uploads_lock:
204             _uploads[upload_id] = {
205                 "filename": name,
206                 "tmp_path": tmp_path,
207                 "parts": 0,
208                 "bytes": 0,
209             }
210         return {
211             "upload_id": upload_id,
212             "max_part_mb": part_bytes // (1024 * 1024),
213             "next_part": f"/api/upload/{upload_id}/part/0",
214         }
215
216
217     @router.put("/api/upload/{upload_id}/part/{index}")
218     async def upload_part(upload_id: str, index: int, request: Request):
219         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
220         _, max_bytes, part_bytes, _ = _upload_cfg()
221         with _uploads_lock:
222             st = _uploads.get(upload_id)
223         if st is None:
224             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
225         if index != st["parts"]:
226             raise HTTPException(
227                 status_code=409,
228                 detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
sequential)",
229             )
230
231         received = 0
232         overflow = None
233         if st.get("writer") is not None:
234             # Remote staging: buffer THIS part (? max_part_mb ? the caps above still abort
an
235             # oversize stream), then hand it to the blocking bucket writer in one threadpool
236             # hop so a network flush never stalls the event loop mid-upload.
237             buf = bytearray()
238             async for chunk in request.stream():
239                 received += len(chunk)
240                 if received > part_bytes:
241                     overflow = (
242                         f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
243                     )
244                     break
245                 if st["bytes"] + received > max_bytes:
246                     overflow = (
247                         f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
248                     )
249                     break
250                 buf += chunk
251             if not overflow and buf:
252                 data = bytes(buf)
253                 await run_in_threadpool(st["writer"].write, data)
254                 st["hasher"].update(data)
255         else:
256             with open(st["tmp_path"], "ab") as fh:
257                 async for chunk in request.stream():
```

```
                    received += len(chunk)
                    if received > part_bytes:
                        overflow = (
                            f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
                        )
                        break
                    if st["bytes"] + received > max_bytes:
                        overflow = (
                            f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
                        )
                        break
                    fh.write(chunk)
        if overflow:
            _abort_upload(upload_id)
            raise HTTPException(
                status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
            )

        with _uploads_lock:
            st["parts"] += 1
            st["bytes"] += received
        return {
            "upload_id": upload_id,
            "received_parts": st["parts"],
            "received_bytes": st["bytes"],
        }


    @router.post("/api/upload/{upload_id}/complete")
    def upload_complete(upload_id: str, req: UploadCompleteRequest):
        """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
        upload_dir, _, _, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if st["parts"] == 0 or st["parts"] != req.total_parts:
            raise HTTPException(
                status_code=409,
                detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
            )
        if st.get("writer") is not None:
            # Remote staging: close() finalizes the streamed object; the md5 accumulated per
            # part IS compute_video_id's digest (same bytes, same algorithm), so
/api/process
            # gets the identity without anyone re-reading the video. (Sync endpoint ?
FastAPI
            # runs it on the threadpool, so the blocking finalize doesn't stall the loop.)
            try:
                st["writer"].close()
            except Exception as e:  # noqa: BLE001 ? surface a clean verdict, then clean up
                _abort_upload(upload_id)
                raise HTTPException(
                    status_code=502,
                    detail=f"Failed to finalize the staged upload ({e}). Re-init to retry.",
                ) from e
            with _uploads_lock:
                _uploads.pop(upload_id, None)
            return {
                "path": st["stage_uri"],
                "video_id": st["hasher"].hexdigest(),
                "size_bytes": st["bytes"],
```

```
318                 "next": "POST /api/process with this path",
319             }
320         final_path = os.path.join(upload_dir, st["filename"])
321         if os.path.exists(
322             final_path
323         ):  # never overwrite an earlier upload with the same name
324             stem, ext = os.path.splitext(st["filename"])
325             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
326         os.replace(st["tmp_path"], final_path)
327         with _uploads_lock:
328             _uploads.pop(upload_id, None)
329         video_id = compute_video_id(final_path)
330         return {
331             "path": os.path.abspath(final_path),
332             "video_id": video_id,
333             "size_bytes": os.path.getsize(final_path),
334             "next": "POST /api/process with this path",
335         }
336 
337 
338     @router.delete("/api/upload/{upload_id}")
339     def upload_abort(upload_id: str):
340         """Abort an in-flight upload and delete its partial file."""
341         with _uploads_lock:
342             known = upload_id in _uploads
343         if not known:
344             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
345         _abort_upload(upload_id)
346         return {"status": "aborted", "upload_id": upload_id}
347 
```

10. assistant (2026-07-08T06:37:39.444Z)

Let me verify the GCS md5 conversion and confirm the current-topology immutability holds.

11. user (2026-07-08T06:37:42.325Z)

```
24:def _md5_hex(md5_b64: "str | None") -> Optional[str]:
25:    """GCS reports ``md5Hash`` as base64 of the raw digest; ``compute_video_id`` (the DB key)
27:    have no md5Hash ? None; a malformed value is treated as absent, never raised."""
28:    if not md5_b64:
31:        raw = base64.b64decode(md5_b64, validate=True)
83:            md5=_md5_hex(getattr(b, "md5_hash", None)),
```

12. user (2026-07-08T06:37:43.586Z)

```
28:**The hard question this doc must resolve (compute topology).** "Upload directly to the L4"
needs an **inbound endpoint on the GPU box**, but the current L4 is a Cloud Run **Job** (batch,
no inbound HTTP ? `backend/compute/cloud_run.py`, dispatched per-request, `[verified]`). So the
epic forces a choice between **A. Cloud Run Service-with-GPU**, **B. GCE L4 VM**, and **C. a
hybrid** presign/redirect. §4 answers it with numbers.
119:**Option C ? Hybrid: control-plane presigns/redirects the upload to a just-spun-up L4,
pipeline runs there, async copy to bucket.**
```

13. assistant (2026-07-08T06:38:09.449Z)

I have enough to render a verdict.

Verdict: REFUTED

The finding is a documentation-completeness observation that is *technically accurate* but does **not** meet the bar of "a real defect a maintainer must fix." There is no concrete failing input/state in the actual code ? the L6 claim is **true** for the shipped topology, and the finding's failure scenario is a purely hypothetical future path that the doc itself recommends

against.

Evidence checked:

1. **The L6 claim (line 41) is correct for the current topology ? it never diverges.** For `trusted_video_id == compute_video_id(fetched_bytes)` to hold, the gs:// object must be byte-identical between the submit-time `stat` (`backend/orchestration.py:231`) and the drain fetch (`backend/orchestration.py:771` `acquire_local_input`). It is, because the object is immutable in-flight by construction:
   - Uploads are **server-mediated** ? parts stream through the control plane (`backend/api/uploads.py:233-254`); the client never gets bucket-write credentials.
   - The staging key is **unique per upload** ? `stage_uri = f"{staging['root']}/uploads/{upload_id}/{name}"` with `upload_id = uuid.uuid4().hex` (`backend/api/uploads.py:168,175`). No two uploads share a path and there is no overwrite API.
   - The `video_id` returned is the streamed whole-file md5 (`backend/api/uploads.py:316`), and `submit_time_video_id` reads back that same object's `md5Hash` via `_md5_hex` (`backend/storage/gcs.py:24-31,83`), returning `None` for composite/absent md5 (? drain re-hashes). So there is no present input where the code produces a wrong DB key.

2. **The reviewer's premise slightly misreads the doc.** Option C (line 119) is "control-plane presigns/redirects the upload **to a just-spun-up L4**" ? a presign to the GPU box, not a *presigned direct-to-bucket client write*. Neither the code nor the doc contemplates a client-writable bucket path today. §4.4 (line 140) explicitly keeps the upload endpoint on the control plane, which is exactly the immutability guarantee the reviewer wants stated ? it is present, just not restated at L6 as a named precondition.

3. **The doc already recommends *against* the only direction that could break this.** L5 (line 40) and §4.3 Option C (line 122) reject upload-onto-L4; §4.5 (line 149-156) already pins durability semantics "needed *only* if a future upload-onto-L4 path is ever adopted." A future engineer wiring client bucket-writes is already routed into a "new ADR / §4.5 override" gate, not silently into win #2.

The prior reviewer self-labeled this a "nit," and that is correct: adding the immutability clause would be a reasonable optional clarification to §6/L6, but the claim is not false, no code path fails, and nothing here blocks a maintainer from merging. It is a style/completeness preference, not a must-fix defect.

14. user (2026-07-08T06:38:16.112Z)

Structured output provided successfully